

SparseCraft: Agentic Hardware-Software Co-Optimization for Sparse Computing

Rajatabha Chakraborty*, M P Samarth*, Vedant Pahariya*, and Priyesh Shukla, *Member, IEEE*
 International Institute of Information Technology, Hyderabad, India
 email: priyesh.shukla@iiit.ac.in

Abstract—Sparse-accelerator design spaces are usually searched against analytical models, so a design point is admitted on what a model predicts rather than on what the hardware does. SparseCraft closes that gap with a language model inside a closed CHIA loop. In each of 15 iterations the model reads the measured outcome of the previous one and edits the Chisel RTL, the memory configuration and the sparse-kernel schedule of a Gemmini accelerator through MCP tool servers, and no candidate counts until it has been checked for legality, elaborated, simulated cycle-accurately, checked bit-for-bit on every output against a golden reference, and synthesised. The harness turns each measurement into the next work order, a diagnosed bottleneck with matching strategy guidance, the history of tried designs and a score of the model’s own prediction, and a second model repairs changes that fail a gate. On a 512×512 GraphChallenge sparse-DNN layer the loop reaches $2.1 \times$ fewer cycles, $9.8 \times$ less off-chip traffic and 22.8% less area than the block-sparse Gemmini baseline, with $5.61 \times$ higher modelled perf/W and $11.8 \times$ lower EDP. The levers span three layers: a schedule that keeps the dense operand resident removes $9.8 \times$ of the traffic, a zero-gated MAC and a zero-row skip unit that the model wrote in Chisel cut energy, and resizing the memories cuts area.

Source-Code: sparsecraft-source-code

Index Terms—LLM agents, hardware/software co-design, sparse accelerators, Gemmini, CHIA.

I. INTRODUCTION

Language models write hardware description code that compiles and passes simulation [1], [2], [3], [4], set accelerator parameters [5], [6] and optimise accelerator kernels [7], but mostly generate once, are scored by an analytical model, or tune one layer. A co-design loop asks more: the model must read a *measured* outcome, change RTL, memory or software, and live with the consequences over many iterations, while the evaluation stays out of its reach. Sparsity makes the problem cross-layer: a block-sparse kernel skips the empty half of our workload’s blocks, but the occupied ones are only 6.25% dense, so 8,388,608 MACs are issued for 524,288 useful ones, and recovering that waste needs RTL, memory and schedule changes together.

SparseCraft is such a loop, built on CHIA [8], an open-source framework for agentic hardware/software co-design, for sparse matrix multiply on Gemmini [9], a Chisel-generated systolic array in Chipyard [10]. The LLM is a node inside the

loop (Fig. 1): each iteration begins with the model reading the diagnosis, counters and history the previous iteration produced, and ends with a measured design that the harness admits or rejects. No human proposes, edits or selects a design, and the harness, energy model and benchmark are fixed and hashed. We contribute:

- a closed loop in which a sealed LLM proposer drives RTL, memory and schedule changes from measured feedback through structured prompts, a second LLM repairs failed changes, and a gate ladder admits a design only after it is built, simulated, checked on every output and synthesised (Section II);
- a 15-iteration run in which every number traces to a published record, with its rejections and regressions (Section IV).

II. THE CLOSED LOOP

A. Structure

Algorithm 1 lists one run; its stages are the nodes of Fig. 1. Iteration 1 evaluates the unmodified baseline. Each later iteration first asserts that the immutable harness inputs (legality rules, Pareto admission, energy model, metrics parser and kernel) still hash to their recorded values (N74). The proposer then edits the tree and the harness evaluates it. An admitted design becomes the parent of the next iteration; a rejected one is rolled back exactly, RTL included, and recorded with the reason it failed. The budget is fixed at 15 iterations, so the counter, not the model, ends the run.

B. The proposer

The proposer (N10) is Claude Opus 5 [11] running as a Claude Code agent [12]; one call is one agentic session of several tool calls. Every built-in tool is removed, so its only actions are three MCP tools served by CHIA: a bash tool inside the Chipyard build container, restricted to three writable files (the design-point parameters, the PE and the zero-bitmap unit), a status tool that returns the last measured design, and a history tool that returns every design tried so far with its result.

C. Prompt structure

The prompts make the model reason in a fixed order. Each *system prompt* defines one role and is composed from shared fragments (writable set, execution rules, feedback schema,

*These authors contributed equally to this work.

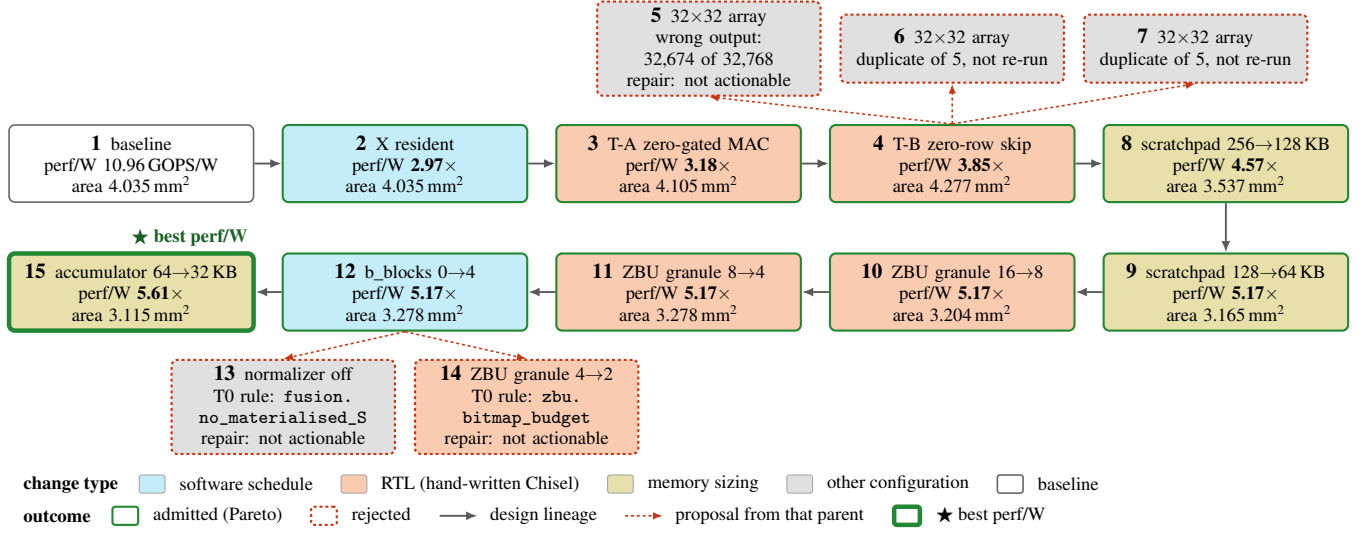


Fig. 2. **Every iteration, along the design lineage.** Admitted designs (solid green) form the serpentine, each the parent of the next; rejected proposals (dotted red) hang from the parent they were proposed from. Blocks give perf/W relative to iteration 1 and area.

on the host, so a scheduling bug cannot appear on both sides of the comparison. Synthesis (N52) runs yosys [17] and OpenSTA [18] on NanGate45 in parallel with simulation. Area is the synthesised logic area plus an estimated SRAM area, every design is evaluated at a fixed 2.0 ns clock, and energy (N53) is modelled from the measured counters.

F. Admission, diagnosis and repair

N60 admits a design that no admitted design matches or beats in time, energy and area (Pareto non-dominance). N61 labels the bottleneck (memory, compute, idle or balanced) from the DMA-wait and array-activity counters, and the label selects the strategy modules of the next work order. A failed gate is classified (N71); infrastructure faults never reach a model. The repairer (N73) receives the shrunk evidence (N72), the proposer’s stated mechanism, both design states

and earlier attempts, and returns a fixed tree, which re-runs the whole ladder, or reports NOT ACTIONABLE with a root cause for the next proposer. Reverting repairs are detected, and attempts are capped at three per iteration.

III. EXPERIMENTAL SETUP

Workload. $Y=AX$ with A the 512×512 upper-left block of the first layer (n1024-11.mtx) of the GraphChallenge sparse DNN [19]: 8,192 nonzeros (3.125%), 512 of 1,024 16×16 blocks occupied; X is dense 512×64, INT8. Values are drawn from a seeded generator and never zero, which keeps the equivalence check sensitive to indexing errors.

Baseline. Iteration 1 is stock Gemmini (16×16 weight-stationary array, 256 KB scratchpad, 64 KB accumulator) running the block-sparse kernel, so every gain is on top of the block walk.

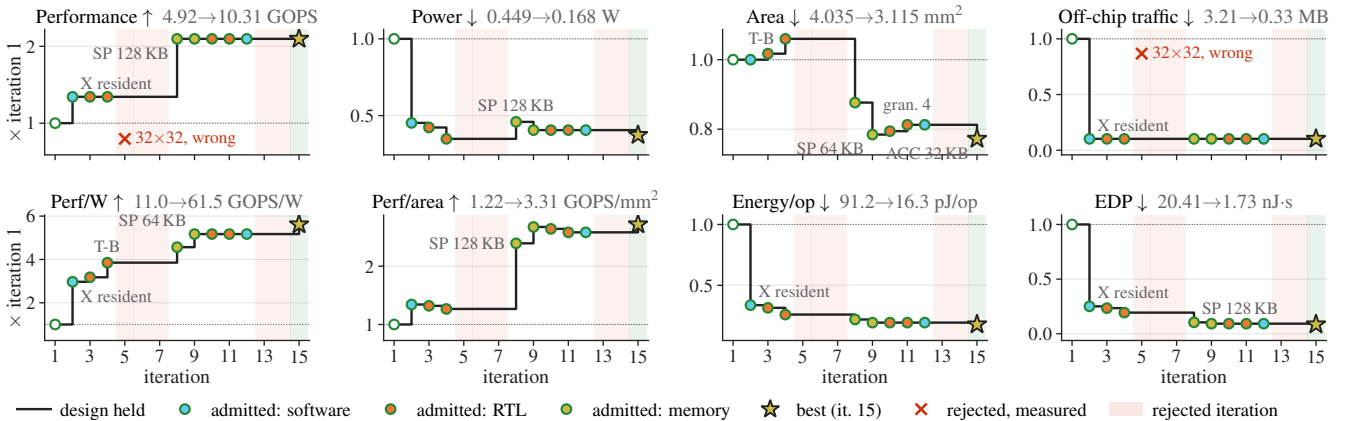


Fig. 3. **Metric evolution**, normalised to iteration 1; titles give the absolute values at iterations 1 and 15. The step line is the design held after each iteration.

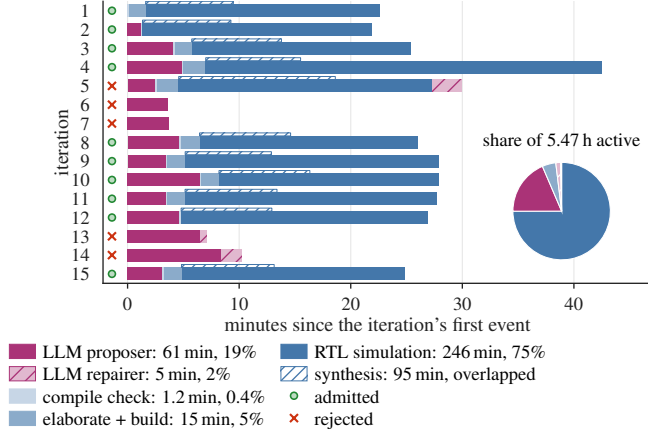


Fig. 4. Wall-clock profile of the loop, from the CHIA profile log.

Metrics. With $W=2\text{nnz}(A)N=1,048,576$ useful operations, t the cycle count times 2.0 ns, E the modelled energy and A the area, we report performance W/t , power E/t , perf/W W/E , perf/area $W/(tA)$, energy/op E/W and EDP= Et , normalised to iteration 1.

IV. RESULTS

A. Trajectory

Fig. 2 shows every iteration and Fig. 3 the design held after each. The model's first move was in software: keeping all of X resident in the scratchpad (iteration 2) cut off-chip traffic $9.8\times$ and cycles $1.34\times$ at zero area. The two RTL mechanisms followed: T-A (iteration 3) cost 1.7% area for 6.6% less energy and T-B (iteration 4) 4.2% area for 17.5% less, neither changing a cycle. With X resident the scratchpad was over-provisioned, and halving it (iteration 8) cut cycles from 79,433 to 50,862 (36%), which the model had not expected; a second halving and a smaller accumulator (iterations 9 and 15) cut area further, while finer ZBU granules (iterations 10, 11) and a schedule change (12) did not improve perf/W.

Five proposals were rejected. A 32×32 array (iteration 5) compiled and simulated but was wrong in 32,674 of 32,768 outputs, since the kernel's 16×16 blocking admits no other array size; iterations 6 and 7 repeated it (duplicates), and 13 and 14 failed legality. The model's directional predictions held in 30 of 36 cases.

B. Time breakdown

The run took 328 minutes of active time (21.9 per iteration, Fig. 4). Charging each instant to the highest-priority stage running, RTL simulation takes 75% and the 17 LLM sessions 20%. Synthesis (95 minutes) always ran beside a simulation, off the critical path, and rejected iterations cost only their LLM time, except iteration 5.

TABLE I
BASELINE (ITERATION 1) AGAINST THE BEST DESIGN (ITERATION 15); NEITHER PRODUCES A WRONG OUTPUT. M: MEASURED; M+E: MEASURED LOGIC PLUS ESTIMATED SRAM AREA; M: MODELLED.

metric		it. 1	it. 15	change
cycles	M	106,650	50,862	$2.1\times$ fewer
off-chip bytes	M	3,211,264	327,680	$9.8\times$ less
area (mm^2)	M+e	4.035	3.115	-22.8%
energy (μJ)	m	95.67	17.04	$5.61\times$ less
power (W)	m	0.449	0.168	$2.68\times$ less
performance (GOPS)	M	4.92	10.31	$2.1\times$
perf/W (GOPS/W)	m	10.96	61.5	$5.61\times$
perf/area (GOPS/ mm^2)	M+e	1.22	3.31	$2.72\times$
EDP (nJ-s)	m	20.41	1.73	$11.8\times$ less

V. CONCLUSION

SparseCraft places a language model inside a closed CHIA loop, where over 15 iterations it co-designed the RTL, memory configuration and software schedule of a sparse Gemini accelerator, driven by measured feedback rather than an analytical model. The loop seals the model to three writable files and a few MCP tools, structures its reasoning through composed prompts, work orders built from measured feedback and parsed predictions, and admits a design only after legality, compilation, elaboration, cycle-accurate simulation, bit-exact equivalence on every output and synthesis, with a second model repairing changes that fail a gate. Working across three layers, the model kept the dense operand resident in software, wrote a zero-gated MAC and a zero-row skip unit in Chisel, and resized the memories, reaching $2.1\times$ fewer cycles, $9.8\times$ less off-chip traffic, 22.8% less area, $5.61\times$ higher modelled perf/W and $11.8\times$ lower EDP than the block-sparse baseline, with its directional predictions right in 30 of 36 cases. Every reported number comes from a design that was built, simulated, checked on every output and synthesised.

ACKNOWLEDGEMENT

This manuscript was drafted with AI assistance, per the venue's policy; the authors are responsible for all claims.

REFERENCES

- [1] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for Verilog code generation," in *Proc. IEEE/ACM ICCAD*, 2023, pp. 1–8.
- [2] J. Blocklove, S. Thakur, B. Tan, H. Pearce, S. Garg, and R. Karri, "Automatically improving LLM-based Verilog generation using EDA tool feedback," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 30, no. 6, pp. 1–26, 2025.
- [3] C.-T. Ho, H. Ren, and B. Khailany, "VerilogCoder: Autonomous Verilog coding agents with graph-based planning and AST-based waveform tracing tool," in *Proc. AAAI*, 2025, pp. 300–307.
- [4] J. Niu *et al.*, "ReChisel: Effective automatic Chisel code generation by LLM with reflection," in *Proc. 62nd ACM/IEEE DAC*, 2025, pp. 1–7.
- [5] Y. Fu *et al.*, "GPT4AIGChip: Towards next-generation AI accelerator design automation via large language models," in *Proc. IEEE/ACM ICCAD*, 2023, pp. 1–9.
- [6] H. Wang *et al.*, "LLM-DSE: Searching accelerator parameters with LLM agents," arXiv:2505.12188, 2025.

- [7] C. Hong, S. Bhatia, A. Cheung, and Y. S. Shao, “Autocomp: A powerful and portable code optimizer for tensor accelerators,” arXiv:2505.18574, 2025.
- [8] A. Cui *et al.*, “CHIA: An open-source framework for principled, agentic AI-driven hardware/software co-design research,” arXiv:2606.27350, 2026.
- [9] H. Genc *et al.*, “Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration,” in *Proc. 58th ACM/IEEE DAC*, 2021, pp. 769–774.
- [10] A. Amid *et al.*, “Chipyard: Integrated design, simulation, and implementation framework for custom SoCs,” *IEEE Micro*, vol. 40, no. 4, pp. 10–21, 2020.
- [11] Anthropic, “System card: Claude Opus 5,” Jul. 2026. [Online]. Available: <https://www.anthropic.com/claude-opus-5-system-card>
- [12] Anthropic, “Claude Code overview,” 2026. [Online]. Available: <https://code.claude.com/docs/en/overview>
- [13] Y.-H. Chen, J. Emer, and V. Sze, “Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks,” in *Proc. 43rd ISCA*, 2016, pp. 367–379.
- [14] Y. N. Wu, P.-A. Tsai, A. Parashar, V. Sze, and J. S. Emer, “Sparseloop: An analytical approach to sparse tensor accelerator modeling,” in *Proc. 55th IEEE/ACM MICRO*, 2022, pp. 1377–1395.
- [15] W. Snyder *et al.*, “Verilator.” [Online]. Available: <https://verilator.org>
- [16] E. Qin *et al.*, “Extending sparse tensor accelerators to support multiple compression formats,” in *Proc. IEEE IPDPS*, 2021, pp. 1014–1024.
- [17] C. Wolf, “Yosys Open SYnthesis Suite.” [Online]. Available: <https://yosyshq.net/yosys/>
- [18] J. Cherry, “OpenSTA: Parallax static timing analyzer.” [Online]. Available: <https://github.com/parallaxsw/OpenSTA>
- [19] J. Kepner *et al.*, “Sparse deep neural network graph challenge,” in *Proc. IEEE HPEC*, 2019, pp. 1–7.